No	Name	Idea Title	Business Problem	Dataset	Target Users	Environment	How to Implement	VOTES
1	Zhoujie	Next-Day Stock Movement Prediction	1. A small asset manager wants to know each day whether its 10 US stocks will close higher tomorrow. 2. We build a system that predicts this after each market close. 3. Decision support only, no trading. 4. Success: signals within 30 minutes of close, reliable pipeline, drift caught within a day. Accuracy only needs to beat "always up".	1. Yahoo Finance via yfinance. 10 stocks including AAPL, MSFT, NVDA, JPM, SPY. 2. Daily prices since 2015, about 25,000 rows. 3. Label: 1 if tomorrow's close is higher, else 0. 4. Features: past returns, moving averages, RSI, volatility, volume, SPY return. 5. Split by time, not randomly. Raw data saved daily so results can be reproduced. 6. Assumptions: fixed stock list. Yahoo stands in for a paid vendor.	1. Traders: read a daily signal table before the morning meeting. 2. ML engineers: watch the pipeline in Airflow, MLflow and Grafana, approve new models. 3. Compliance: trace which model made which prediction. 4. Batch once a day, plus a small REST API for ad-hoc lookups.	1. Runs locally with docker-compose, one command. 2. Stack: Airflow, PostgreSQL, MinIO, MLflow, FastAPI, Evidently, Grafana. 3. Cloud target is AWS. Swap S3 for MinIO, RDS for PostgreSQL. Config change only. 4. CPU only. Training takes seconds. 5. GitHub with PR review and GitHub Actions.	1. One container per step: ingest, features, train, batch predict, monitor. 2. Daily Airflow DAG runs the pipeline. A retrain DAG fires on drift or accuracy drop. 3. LightGBM tracked in MLflow, registered in Model Registry. 4. Shadow deployment: new model runs beside the old one for 5 days, then promoted if not worse. 5. Why: batch fits the daily cycle, LightGBM is fast and explainable, shadow fits batch jobs.	
2	Abinav	Dengue cluster early Warning*	NEA has limited inspection manpower and can't fumigate/inspect every estate every week Without prediction, they react to clusters after they form rather than pre-empting them Goal: predict which subzones are likely to develop new clusters in the coming week, so inspections are prioritised there	NEA Dengue Clusters (GeoJSON, data.gov.sg) - cluster location, case size, updated regularly MOH Weekly Dengue Cases (data.gov.sg) - historical weekly counts, 2014–2018 confirmed available, check for more recent years Weather data (rainfall, temperature) from data.gov.sg or Meteorological Service Singapore - dengue is seasonal/weather-linked Optional: URA subzone boundaries to join cluster locations to standard geographic units	NEA vector control officers deciding where to send inspection teams Town councils, who could use it to guide local clean-up campaigns	"	Based on scope lvl1: time-series or regression model forecasting case counts per subzone for the next week lvl 2: add weather features, lag effects (rainfall 2–3 weeks prior correlates with cases) lvl3: spatial model that accounts for clusters spreading to neighbouring subzones, or a dashboard simulating "what if we inspect subzone X now"	
3	Mike	Dynamic Airline Ticket Pricing	1. Airline ticket prices need to change according to demand and market conditions. 2. Fixed pricing may result in empty seats or missed revenue opportunities. 3. The airline wants to predict expected demand and recommend an appropriate ticket price. 4. Pricing should consider booking time, remaining seats, route, season, flight time and competitor prices. 5. Goal: maximize expected revenue while maintaining competitive fares.	1. Historical flight booking / airfare dataset from Kaggle. 2. Features: airline, origin, destination, journey date, booking date, departure time, duration, stops and ticket price. 3. Additional features can include days before departure, remaining capacity and historical demand. 4. Target: expected ticket demand / ticket price. 5. Kaggle flight-price datasets are available for regression modelling; one dataset contains airline, route, timing, duration, stops and price information. https://www.kaggle.com/datasets/nileshiq/flight-fare/data	1. Airline revenue managers: monitor demand and adjust fares. 2. Pricing analysts: analyze price elasticity and competitor pricing. 3. Airline management: monitor revenue performance. 4. E-commerce / booking platform: provide dynamic fare recommendations.	1. Python / Jupyter Notebook / Google Colab. 2. Regression, XGBoost / LightGBM for prediction. 3. MLflow for experiment and model tracking.	1. Clean flight and booking data. 2. Perform EDA on demand and ticket prices. 3. Engineer features such as days-to-departure, season, route demand and remaining capacity. 4. Train regression models to predict expected demand/price. 5. Estimate price elasticity using historical price-demand relationships. 6. Simulate different ticket prices and estimate expected revenue. 7. Build a pricing recommendation API/dashboard. 8. Retrain periodically as demand patterns change.	
4	Zack	Dark-fleet monitoring	Sanctioned Iranian and Russian oil moves on a "shadow fleet" of old tankers. They switch off their AIS transponders, transfer oil between ships at sea, and keep changing names and flags. One hotspot is the Eastern Outer Port Limits, about 70 km off Johor. Shadow fleet transfers off Peninsular Malaysia reportedly more than doubled in a year. Sanctions lists only name ships that have already been caught, and checking ships by hand doesn't scale. Companies in Singapore's shipping and oil-trading hub that insure, charter or trade with a sanctioned ship face heavy penalties. They need an early warning for ships that behave like sanctioned ones before those ships are listed.	1.Live ship tracking: aisstream.io, a free real-time AIS feed. You subscribe to a map rectangle covering the Singapore Strait and the Johor transfer zone, and keep tankers only. 2. Labels: OpenSanctions vessel data, free for non-commercial use. It lists ships sanctioned by the US (OFAC), EU, UK and UN by IMO number, and keeps their past flags. Analysts' confirm/dismiss decisions become extra labels.	Primary: sanctions compliance analysts at marine insurers, oil traders and ship charterers in Singapore. Secondary: port and maritime authorities deciding which ships to inspect.	Stack: Airflow for scheduling the pipeline MinIO and Postgres for storage Great Expectations for data validation Feast as the feature store MLflow for experiment tracking and the model registry FastAPI for the online API Evidently for drift and bias monitoring Prometheus and Grafana for API metrics GitHub Actions for CI Compute: CPU only. The model is XGBoost on one table with a row per ship. Targets: the watchlist is ready by 8am Singapore time each day, and the API answers in under a second.	Pull live ship-tracking data and the sanctions list every day (Airflow, stored in MinIO). Turn raw positions into events: transponder gaps, ship-to-ship meetings, loitering. Build per-ship features (Feast) and train an XGBoost model against known sanctioned ships (MLflow). Serve a nightly watchlist plus a FastAPI single-ship lookup, testing new models in shadow mode first. Monitor drift, bias and accuracy (Evidently), and retrain when new sanctions rounds arrive.	
5	Zhoujie	AML Alert System: Finding Suspicious Bank Accounts with ML	Banks use fixed rules to detect money laundering, which causes too many false alarms. Investigators have limited time, so they need to see the riskiest accounts first.	https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml	AML investigators get a daily ranked list of risky accounts, review them, and mark each one as real laundering or a false alarm.		Airflow runs a daily pipeline to ingest data, build features, score accounts and send alerts. We train with XGBoost, manage models with MLflow, serve them with FastAPI, and monitor drift with Evidently.	
6	Alekhya	Service ticket routing						
7	Alekhya	Item defect identification						
8	Joyce	EV Predictive Maintenance & Battery Health Monitoring	EV operators currently rely on scheduled maintenance or reactive repairs. Develop an ML system to identify deteriorating vehicle/battery conditions early, allowing maintenance teams to intervene before failures or severe degradation occur.	EVIoT-PredictiveMaint Dataset (Kaggle) — EV operational/IoT data including SOC, SOH, battery voltage/current/temperature, charge cycles, motor temperature, vibration, torque and RPM.	EV fleet operators, maintenance engineers, workshop planners, asset managers			